

On the Correctness of Suffix Tree Construction Through Ukkonen's Algorithm Without Suffix Links

A Practical Commentary

My Name

September 30, 2026

1 Introduction

The following is a correctness commentary on my very own implementation of Ukkonen's algorithm on suffix tree construction. Ukkonen's is uniquely known for it's online capabilities, formalised by Esko Ukkonen in On-line construction of suffix trees.

Suffix trees themselves were introduced in Linear Pattern Matching Algorithms, as a way to reduce basic string matching in reduced time.

Ukkonen's algorithm went on to become the first online construction of the suffix tree, which allows for usage whilst being created. My implementation opts for a more naive approach than most implementation, where instead of being upper bound by linear time, it faces $O(n^2)$. As with all tradeoffs, there is one such advantage that the loss of suffix arrays will reduce space overhead of the algorithm. However, this is not noticeable enough to justify its usage.

2 A Commentary On My Implementation of Ukkonen's Algorithm

2.1 Pre-condition

Let Σ be the set of all characters in a language where $\$ \notin \Sigma$. There must be some string s , such that $s \in \Sigma^*$, also represented as $s = \{s_0, s_1 \dots s_n\}$.

2.2 Post-condition

A suffix tree exists such that the root node R contains child nodes all of unique suffixes.

2.3 Initialisation

Let $\alpha(n)$ return the starting index of the ingoing edge into node n , and $\beta(n)$ return the exclusive ending index of the ingoing index.

Let the root of our suffix tree be some node R

Invariant 1 - Let root node R exist such that $\alpha(R) \cup \beta(R) = \emptyset$

The root node is initialised without usage.
continuing

Invariant 1 - $Parent(A) \in nodes$

Invariant 2 - $Edge(A) \in nonrootnodes$

Invariant 3 - $Length(w) \geq Length(A) \geq 0$

The remainder exists such that

Invariant - $Length(w) \geq rem \geq 0$

2.4 Global Index Pointer and the Loop, Extension 1

Invariant - $Length(w) \geq i \geq 0$

2.5 Creating a new Node, Extension 2.i

Invariant - *There can never be 2 nodes of the same first character in parent*

2.6 Using Existing Node

Invariant - $Length(w) \geq i \geq 0$

2.7 Splitting of the Current Edge, Extension 2.ii

Invariant - $Length(w) \geq i \geq 0$

2.8 Extension Rule 3

Invariant - Holds our loop invariant